

PART VII — PRODUCTION, DEPLOYMENT, AND BEYOND

Chapter 41B

Productionizing Document Conversion: The Marker Microservice and Switchable Ingestion

“An evaluation utility earns production status only once someone builds the boundary that lets it run safely next to everything that already depends on a different set of dependencies.”

Chapter 41B — Productionizing Document Conversion: The Marker Microservice and Switchable Ingestion

Chapter 5B evaluated Docling, Unstructured, and Marker-PDF as document-conversion engines and ended with a deliberate non-decision: ADR-072 kept all three as evaluation utilities rather than declaring any of them the authoritative ingestion path, explicitly deferring adoption pending a downstream retrieval-quality bake-off. This chapter picks that thread back up once the bake-off's answer arrived — Marker's conversion quality was worth production adoption — and covers the genuinely harder problem that decision created: Marker's own dependency stack cannot simply be installed into ``app_workflow/`` alongside everything else already running there.

A note on sourcing before this chapter goes further: ``marker_service/``, the GPU microservice this chapter describes, does not exist as tracked files in this repository's current snapshot — no ``server.py``, ``Dockerfile``, or ``docker-compose.yml`` are present to read directly, and git history confirms the directory has never been committed to this particular checkout. Everything this chapter states about its internals is sourced from three real, tracked artifacts instead: ADR-073's decision record, the 2026-07-24 entries in ``docs/Status.md`` and ``docs/Architecture.md``, and the "Marker PDF Service" section of this project's own ``README.md``. Where this chapter would normally quote source code directly, it quotes these ledger and README sources instead, and says so.

41B.1 From evaluation to production

Chapter 5B.9 closed with a specific, careful phrase this chapter is now explicitly revisiting: raw converter Markdown was "kept as an evaluation utility, not the authoritative representation." That phrasing was deliberate rather than tentative — it left the door open for exactly the reversal this chapter documents, without pre-committing to it before real retrieval-quality evidence existed. ADR-072 is the record of that original caution, and ADR-073 and ADR-074 are the pair of decisions that walked through it once a Marker-backed loader was judged ready to wire into ``app_workflow/`` for real. The acceptance-gate checklist Chapter 5B.9 sketched — and the ``assess_conversion()`` function skeleton it left unimplemented — remain unbuilt as a formal automated gate; what changed is that a human judgment call, backed by the research this project's ledgers document, decided Marker had cleared that bar in practice.

It is worth being precise about what "a bake-off decided the question" means here, because it is easy to over-credit this decision with more rigor than it had. No formal ``assess_conversion()`` scoring run produced a number that crossed a threshold; the decision was a human synthesis of the qualitative comparison Chapter 5B ran — Docling, Unstructured, and Marker-PDF evaluated on text-retention, structural fidelity, and downstream retrieval outcomes — combined with the practical reality that Marker's output was the one worth the engineering cost of a dedicated microservice to unlock. A reader building the same pipeline should treat this chapter's adoption as a case study in a real, defensible judgment call made without a fully automated gate, not as proof that a formal gate is unnecessary.

41B.2 Why Marker can't simply be pip-installed into app_workflow/

Research topic 62's dependency-conflict analysis is the direct source for this section, and its finding is sharper than "these libraries happen to be untested together": Marker's own stack requires a newer floor version of `transformers` and `pillow` than `sentence-transformers` — the embedding backbone Chapter 8 covers and every retrieval call in this project depends on — can tolerate on Python 3.14. This is not a version-pinning inconvenience solvable by picking slightly different pinned versions; it is a genuine floor conflict, where the oldest version Marker will run on is newer than the newest version `sentence-transformers` will run on in the same interpreter.

Figure 41B.1 draws the shape of the conflict and the boundary that resolves it. The resolution this project chose was not to pick a side and accept the other library's degraded compatibility — it was to refuse to run both in the same Python process at all.

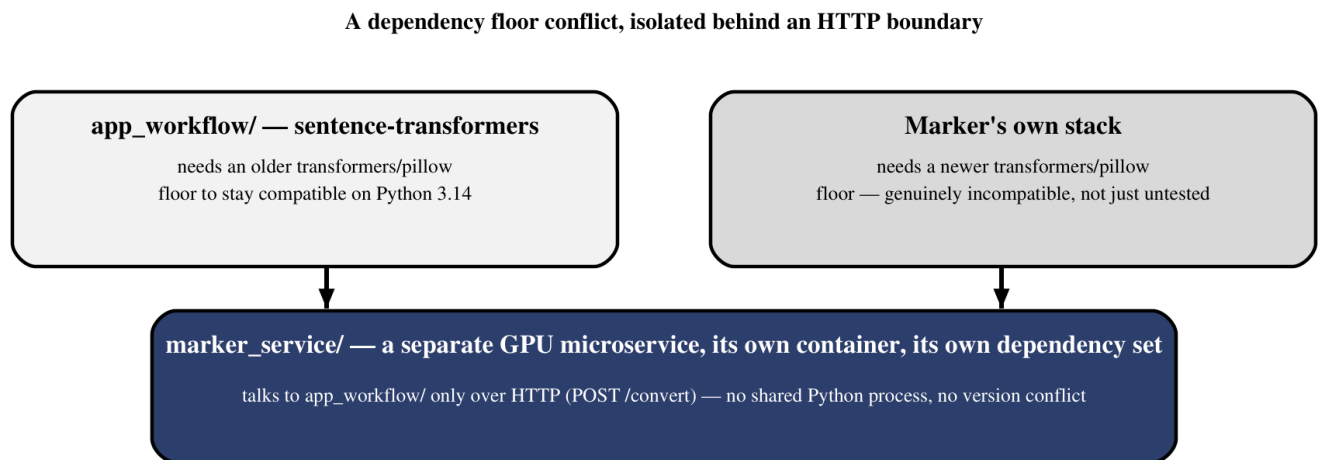


Figure 41B.1 — Two genuinely incompatible dependency floors, resolved by removing the shared process rather than by choosing a compromise version.

41B.3 Isolating Marker as a GPU microservice

Per ADR-073 and the Architecture ledger's description, `marker_service/` runs as a standalone FastAPI application exposing exactly two routes: `POST /convert` and `GET /health`. The project's own `README.md` documents the contract precisely — `GET /health` returns `{status, ready, cuda}`, and `POST /convert` returns `{markdown, source, chars}`. A single `PdfConverter` instance is built once at process boot, dispatched through `asyncio.to_thread` so the boot-time model load doesn't block the FastAPI event loop, and every subsequent conversion request is serialized behind a `threading.Lock` — Marker's underlying model is not safe for concurrent use from multiple requests at once, so the lock trades conversion throughput for correctness rather than risking two simultaneous conversions corrupting each other's state.

41B.4 What the container must reproduce that the host got for free

Research topic 62 and the 2026-07-24 architecture entry both stress that this microservice's Dockerfile is not a generic Python container — it has to rebuild several things a developer's host machine already had installed for unrelated reasons. A CUDA 13.0-provenance torch build has to be pinned explicitly rather than relying on whatever torch a base image happens to ship. `download_font()`, a Marker-internal call that normally runs lazily on first use, has to be pre-seeded at build time so a cold container doesn't stall its first real request fetching font assets over the network. The Triton JIT toolchain Marker's model-compilation path depends on needs `gcc` and `libc6-dev` present in the container image — both entirely invisible dependencies on a Windows development host, where neither package exists in any meaningful sense, and both easy to forget when writing a Dockerfile from a Windows-based development environment. Finally, a persistent model-cache volume must be mounted so the (large) downloaded model weights survive a container restart rather than re-downloading on every boot.

41B.5 The pdftext multi-worker abort (BUG-077)

Definition — BUG-077 — pdftext multi-worker abort

Marker's underlying `pdftext` library defaults to a multi-worker extraction pool for PDF text extraction. On PDFs longer than roughly 40 pages, one worker process in that pool would occasionally die, and — because the pool had no fallback for a dead worker — the entire conversion aborted rather than degrading gracefully to the workers that were still alive.

The bug ledger records this as closed, mitigated by configuration rather than by a code change to `pdftext` itself: forcing `pdftext_workers=1` in `marker_service/server.py`'s `_CONVERTER_CONFIG` eliminates the multi-worker pool entirely, trading extraction parallelism within a single conversion for reliability on long documents. This exact setting matches Marker's own CLI and server defaults for single-conversion-at-a-time deployments — this project's fix is not a workaround so much as an alignment with how Marker's own maintainers already recommend running it outside a high-throughput batch context. Worth noting for continuity: the bug ledger records that `server.py`'s own comments informally referred to this issue as "BUG-025," a name collision with an unrelated, differently numbered bug elsewhere in the project — the canonical, correctly disambiguated reference is BUG-077.

41B.6 The five-module switchable ingestion package

ADR-074 records the replacement of the older monolithic `ingest.py` with a five-module package under `app_workflow/ingestion/`: `ingestion_requests.py` (file discovery and the `run_ingestion()` coordinator), `marker_loader.py` (the HTTP client to `marker_service/`), `unstructure_loader.py` (the pre-existing Unstructured-based loader, kept as the default path), `custom_splitter.py` (a Marker-Markdown-aware structural splitter, including table re-serialization logic, at 1,409 lines the largest module in the package), and `recursive_splitter.py` (a smaller, generic recursive text splitter used when the custom splitter is off). Splitting a single large `ingest.py` into five focused modules is what makes Section 41B.7's independent-switch design practical — each axis of the loader × splitter matrix maps to swapping in one module for another, rather than branching deep inside one large function.

41B.7 The loader x splitter matrix

`'ENABLE_MARKER_LOADER'` and `'ENABLE_CUSTOM_SPLITTER'` are two entirely independent boolean flags in `'config.py'`, both defaulting to `'False'`, and `'ingestion_requests.py'`'s `'_resolve()'` function is the small, precise piece of logic that governs how a per-call override interacts with each default:

```
def _resolve(override, default):  
    return default if override is None else bool(override)
```

`'run_ingestion(enable_marker_loader=None, enable_custom_splitter=None)'` calls `'_resolve()'` independently for each flag, meaning a caller can override either one, both, or neither, without the two interacting — turning on the Marker loader does not implicitly turn on the custom splitter, and vice versa. Figure 41B.2 lays the resulting matrix out explicitly.

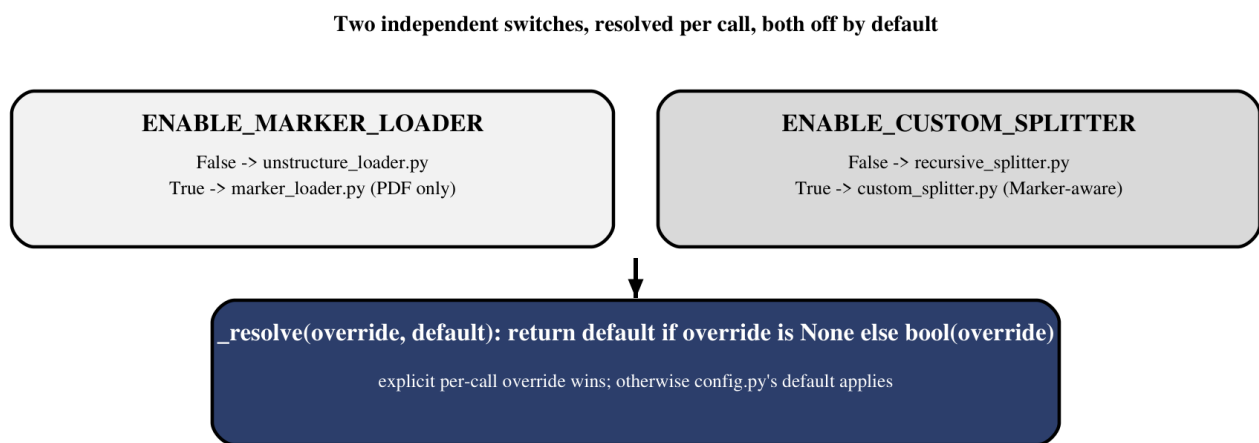


Figure 41B.2 — Two independent switches produce four real ingestion configurations; `_resolve()` is the same three-line logic applied twice, once per axis.

Both flags defaulting to `'False'` is a deliberate conservatism, consistent with Chapter 36B's broader pattern of feature flags shipping off until evidence justifies flipping them: the Unstructured loader and the generic recursive splitter remain the safe, well-exercised default path, and a caller has to explicitly opt into the newer Marker-backed loader or the custom structural splitter rather than inheriting them by default the moment the code merges.

41B.8 In-memory loading end to end

Before this switchable package existed, document conversion routinely persisted intermediate Markdown or JSON files to disk between the conversion step and the chunking step — a reasonable design when conversion was a slow, standalone script run ahead of ingestion. Once Marker runs over HTTP as a request/response service rather than a local script, and once tabular data converts directly to JSON in memory rather than through an intermediate file format, that persisted-intermediate-file step became unnecessary rather than merely inconvenient: `'marker_loader.py'`'s HTTP client receives Markdown text directly in the response body and hands it straight to the splitter, and the entire loader-to-splitter path for a document can run without ever touching disk for anything beyond the original source file and the final vector-store write.

41B.9 The POST /ingest endpoint

``app_workflow/api.py`` exposes ingestion itself as an HTTP endpoint, not just a standalone script. Its ``IngestRequest`` model is deliberately minimal — exactly two optional fields, ``ENABLE_MARKER_LOADER: Optional[bool] = None`` and ``ENABLE_CUSTOM_SPLITTER: Optional[bool] = None`` — mirroring Section 41B.7's per-call override pattern at the API boundary. The handler wraps the entire ``run_ingestion()`` call in ``asyncio.to_thread(...)``, the same pattern Section 41.1 covered for keeping a long-running blocking call from stalling the FastAPI event loop, and returns the run-summary contract ``run_ingestion()`` itself produces: ``files_discovered``, ``documents_loaded``, ``chunks_created``, and ``documents_in_store`` — four numbers that let a caller verify an ingestion run actually processed what it was expected to, without needing to separately query the vector store afterward.

```
{
  "files_discovered": 42,
  "documents_loaded": 42,
  "chunks_created": 613,
  "documents_in_store": 613,
  "marker_loader": true,
  "custom_splitter": true
}
```

The response also echoes back the two resolved boolean flags themselves — ``marker_loader`` and ``custom_splitter`` — which matters for exactly the reason Section 41B.7's ``_resolve()`` logic exists: a caller who sent no override at all still needs a way to confirm which path ``config.py``'s defaults actually routed the request through, rather than assuming its own request payload was the only source of truth for what ran.

41B.10 The PDF-only loader caveat

Common pitfall — Turning on `ENABLE_MARKER_LOADER` against a mixed corpus

``marker_loader.py`` filters its input list to PDFs only, in code that is unambiguous about what happens to everything else: ``_PDF_EXT = {".pdf"}``, followed by ``pdf_paths = [Path(fp) for fp in file_paths if Path(fp).suffix.lower() in _PDF_EXT]``. Every non-PDF file in the batch is silently dropped from that ingestion run — not errored, not logged as skipped in a way that stands out, simply absent from what gets processed.

This matters because a corpus mixing PDFs with ``.docx``, ``.md``, or ``.txt`` files — a realistic shape for many real document collections — will silently lose every non-PDF file the moment ``ENABLE_MARKER_LOADER=True`` is set, with the run-summary's ``documents_loaded`` count being the only signal something was dropped, and even that only if an operator is specifically comparing it against ``files_discovered``. Two honest ways to handle a mixed corpus follow directly from this constraint: fall back to ``unstructure_loader.py`` (the default, non-PDF-restricted path) for the whole batch, accepting Marker's better PDF conversion quality is unavailable this run, or run a two-pass ingestion — one pass with the Marker loader for the PDF subset, a second pass with the Unstructured loader for everything else — accepting the added operational complexity of coordinating two separate ingestion calls against the same corpus.